SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: STUDENT SELF-SERVICE PASSWORD CHANGE

University College of Jaffna

Department of Interdisciplinary Studies

Information and Communication Technology

SCM (K72C001M12)

NVQ Level 6 – Semester 1

Change Request Information

Field	Details
Change Request ID	CR-020
Change Request Title	Student Self-Service Password Change
Student Name	S.Hindusha
Student ID	JF/ICT24_26
Implementation Status	Completed – Testing Pending
Date	13 October 2026
System Version	Student Management System v1.1
Priority	Medium
Difficulty	Medium
Change Type	Frontend and Backend Enhancement

1. INTRODUCTION

1.1 Change Request Summary

The Student Management System required a facility that allows students to change their account passwords independently through the Student Dashboard.

Previously, students did not have a dedicated self-service password change facility. This change introduces a secure password change feature that enables authenticated students to enter their current password, specify a new password, and confirm the new password.

The system validates the submitted information, verifies the current password against the stored password hash, and securely updates the password hash in the database.

The Student Dashboard has also been enhanced with a Change Password button. The password change form remains hidden when the dashboard initially loads and appears only when the student clicks the button. The student can close the form using the Cancel button or the toggle button.

The primary objectives are to improve account security, increase usability, and allow students to manage their passwords independently while preserving existing system functionality.

1.2 Existing System Behaviour

Before implementing CR-020, the system had the following characteristics:

- Students could log in using their registered usernames and passwords.
- Password verification was handled by the existing authentication controller.
- Passwords were stored as hashes in the users table.
- The Student Dashboard displayed student profile information.
- Students could not change their passwords through the Student Dashboard.
- No dedicated form was available for entering the current password, new password, and confirmation password.
- Students depended on existing administrative or account-management procedures when they needed assistance with password changes.

1.3 Objectives of the Change

The objectives of CR-020 are:

- Add a Change Password button to the Student Dashboard.
- Display the password change form only when requested by the student.
- Provide current password, new password, and confirmation fields.
- Validate required fields and password strength.
- Verify the current password before updating the account.
- Ensure that the new password and confirmation match.
- Prevent the new password from being identical to the current password.
- Store the new password as a secure hash.
- Restrict password changes to the authenticated student's own account.
- Display appropriate success and error messages.
- Allow cancellation without changing the password.
- Preserve existing login, logout, student profile, and administrator functionality.

2. IMPLEMENTATION DETAILS

2.1 Implementation Overview

CR-020 was implemented by modifying the existing frontend and backend components of the Student Management System.

The frontend was enhanced with a Change Password button and a new ChangeStudentPassword.jsx component. The component manages the password fields, client-side validation, submission state, and response messages.

The backend was extended with a new authenticated API endpoint:

PATCH /api/students/me/password

The route uses the existing authentication middleware and restricts access to users with the STUDENT role.

The password change controller obtains the authenticated user's ID from the verified request token rather than accepting an arbitrary user ID from the frontend. It validates the submitted values, retrieves the stored password hash, verifies the current password using bcryptjs, hashes the new password, and updates the existing password hash in the database.

The implementation reuses the existing database schema and does not require a new table or column.

2.2 Files Modified and Added

File	Change	Purpose
backend/src/models/userModel.js	Modified	Retrieves the stored password hash and updates the password hash.
backend/src/controllers/studentController.js	Modified	Implements the changeMyPassword controller.

backend/src/routes/studentRoutes.js	Modified	Adds the authenticated student password change route.
frontend/src/components/ChangeStudentPassword.jsx	Added	Implements the password change form and frontend validation.
frontend/src/pages/StudentDashboardPage.jsx	Modified	Adds the Change Password button and form visibility control.
frontend/src/styles/studentDashboard.css	Modified	Adds styling for the button and password change form.

Existing Files Retained

The following existing files are retained without requiring changes for this implementation:

- backend/src/models/studentModel.js
- backend/src/controllers/authController.js
- backend/src/middleware/authMiddleware.js

The existing validation utility is reused where applicable.

2.3 Database Changes

No database schema modification is required.

The implementation uses the existing users table and its password_hash field to store the new password hash.

The relevant database fields are:

Field	**Purpose**
user_id	Identifies the user account.
username	Identifies the account during login.
password_hash	Stores the hashed password.
role	Identifies the account role, such as STUDENT or ADMIN.

The system does not store plaintext passwords in the database and does not introduce a separate password storage table.

2.4 Frontend Implementation

2.4.1 Student Dashboard Modification

The StudentDashboardPage.jsx component was modified to include a state variable named showChangePassword.

The variable controls whether the password change form is visible.

Initially, the state is set to false, so the form remains hidden when the dashboard loads. When the student clicks the Change Password button, the state changes to true and the form is displayed.

Clicking the toggle button again closes the form.

The button label changes according to the current state:

- Change Password — when the form is hidden.
- Cancel Change Password — when the form is visible.

Existing profile information, loading messages, error handling, and Logout functionality are retained.

2.4.2 Password Change Form

A new ChangeStudentPassword.jsx component was created.

The form contains the following fields:

1. Current Password
2. New Password
3. Confirm New Password

The component manages the form values, validation messages, loading state, API request, and response messages.

The Cancel button closes the form without submitting a password change request.

After a successful password change, the component displays a success message and clears the password fields.

2.4.3 Client-Side Validation

Client-side validation provides immediate feedback before the form is submitted.

The validation checks include:

- All required fields must be completed.
- The new password must satisfy the configured password strength requirements.
- The new password and confirmation password must match.
- The new password must not be identical to the current password.

The configured password strength requirements are:

- At least eight characters.
- At least one uppercase letter.
- At least one lowercase letter.
- At least one number.
- At least one special character.

Client-side validation improves usability but does not replace backend validation.

2.5 Backend Implementation

2.5.1 API Route

The following API endpoint was introduced:

HTTP Method: PATCH

Endpoint: /api/students/me/password

The route is protected by the existing authentication middleware and student-role authorization.

Only authenticated users with the STUDENT role are permitted to access the student-specific password change operation.

The route uses the authenticated user's identity from the verified token. It does not accept an arbitrary student ID from the request body.

2.5.2 Password Change Controller

The changeMyPassword controller was added to studentController.js.

The controller performs the following operations:

1. Receives the current password, new password, and confirmation password.
2. Checks that the required fields have been provided.
3. Validates the new password against the configured strength requirements.
4. Confirms that the new password and confirmation password match.
5. Rejects a new password that is identical to the current password.
6. Obtains the authenticated user's ID from req.user.userId.
7. Retrieves the stored password hash for the student's account.
8. Verifies the current password using bcrypt.compare().
9. Hashes the new password using bcrypt.hash().
10. Updates the stored password hash through the user model.
11. Returns an appropriate success or error response.

The controller does not return the stored password hash or expose password values in its response.

2.5.3 User Model Modification

The userModel.js file was extended with two functions.

getPasswordHashByUserId(userId)

Retrieves the stored password hash for the specified student account. The query restricts the operation to accounts with the STUDENT role.

updatePasswordHash(userId, passwordHash)

Updates the password hash for the specified student account.

The model uses parameterized SQL queries to separate supplied values from SQL statements.

Existing user lookup and last-login tracking functions are retained.

2.5.4 API Communication and Error Handling

The frontend submits the password change request through the existing API service.

The request contains the following fields:

- currentPassword
- newPassword
- confirmPassword

The backend independently validates the request before updating the database.

The application handles missing fields, weak passwords, confirmation mismatches, incorrect current passwords, authentication failures, and successful password changes.

A loading state is maintained during submission to help prevent repeated requests while processing is in progress.

2.6 User Interface and CSS Changes

The studentDashboard.css file was updated to style the Change Password button and password change form.

The styling includes:

- Blue button background.
- White button text.
- Rounded corners.
- Hover and active effects.
- Keyboard focus indication.
- Disabled-state styling.
- Responsive button sizing for smaller screens.

The form appears below the student profile when activated. Existing dashboard styling and profile layout are preserved.

3. IMPLEMENTATION WORKFLOW

The password change process follows these steps:

1. The student logs in using the existing authentication process.
2. The Student Dashboard loads the student's profile.
3. The password change form remains hidden by default.
4. The student clicks the Change Password button.
5. The application displays the password change form.
6. The student enters the current password.
7. The student enters the new password and confirmation.
8. The frontend validates the submitted values.
9. If validation fails, an error message is displayed and the request is not submitted.
10. If validation succeeds, the frontend sends a PATCH request to the backend.
11. The backend verifies the authentication token and student role.
12. The backend validates the submitted password information.
13. The backend retrieves the stored password hash for the authenticated student.
14. The backend verifies the current password.
15. If the current password is incorrect, the operation is rejected.
16. If validation succeeds, the backend hashes the new password.
17. The backend updates the existing password_hash field.
18. The frontend displays a success message and clears the password fields.
19. The student can close the form using Cancel or the toggle button.
20. Existing student profile and logout functionality remain available.

4. ERROR HANDLING AND SECURITY

The implementation includes the following validation and security measures:

Condition	Expected Behaviour
Missing fields	Reject the request and display a validation message.
Weak new password	Reject the request without changing the password.
Confirmation mismatch	Reject the request.
Incorrect current password	Reject the request without updating the password hash.
New password identical to current password	Reject the request.
Missing or invalid authentication token	Reject access through the authentication middleware.
Non-student account	Deny access to the student-specific password change route.
Successful password change	Store the hash of the new password.
Form cancellation	Close the form without submitting a password change request.
Database or API error	Display an appropriate error message without exposing sensitive information.

The implementation uses the authenticated student's identity and does not allow a student to select another account by submitting a different user ID.

The backend performs its own validation to ensure that client-side checks cannot be bypassed by directly calling the API.

5. TESTING AND VERIFICATION

5.1 Testing Approach

The implementation should be tested through the frontend, backend API, authentication middleware, and database.

Testing must confirm that the form visibility controls work correctly, password validation is enforced, incorrect current passwords are rejected, valid password changes succeed, and existing application functionality remains operational.

Testing Environment

Item	Details
Environment	Development
Browser	Google Chrome / Microsoft Edge
Frontend	React and Vite
Backend	Node.js and Express.js
Database	MySQL
Authentication	JWT
Password Hashing	bcryptjs
Test Roles	Student and Administrator

The testing types include functional testing, UI testing, client-side validation testing, backend validation testing, authentication and authorization testing, API testing, database testing, error-handling testing, and regression testing.

5.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the Student Dashboard.	Profile is displayed and the password form is hidden.	To be recorded.	Pending
TC-02	Click the Change Password button.	The password form becomes visible.	To be recorded.	Pending
TC-03	Click the toggle button again.	The form closes.	To be recorded.	Pending
TC-04	Click Cancel in the form.	The form closes without submitting a request.	To be recorded.	Pending
TC-05	Submit the form with empty fields.	Required-field validation is displayed.	To be recorded.	Pending
TC-06	Enter a password shorter than eight characters.	The weak password is rejected.	To be recorded.	Pending
TC-07	Enter a password without an uppercase letter.	The password is rejected.	To be recorded.	Pending
TC-08	Enter a password missing a required lowercase letter, number, or special character.	The password is rejected.	To be recorded.	Pending
TC-09	Enter different new password and confirmation values.	The request is rejected.	To be recorded.	Pending
TC-10	Enter an incorrect current password.	The password remains unchanged and an error is displayed.	To be recorded.	Pending
TC-11	Enter a new password identical to the current password.	The request is rejected.	To be recorded.	Pending

TC-12	Change the password using valid credentials and a strong new password.	The password hash is updated and success is displayed.	To be recorded.	Pending
TC-13	Log in using the new password.	Authentication succeeds.	To be recorded.	Pending
TC-14	Log in using the old password after the change.	Authentication fails, provided no other credential change has occurred.	To be recorded.	Pending
TC-15	Submit a password change request without a valid token.	The backend rejects the request.	To be recorded.	Pending
TC-16	Attempt to access the route as an administrator.	The student-only route denies access.	To be recorded.	Pending
TC-17	Verify the stored password after a successful change.	The database contains a password hash, not plaintext.	To be recorded.	Pending
TC-18	Navigate through the dashboard after the change.	Profile and Logout functionality continue to work.	To be recorded.	Pending

5.3 Test Summary

Test Result	Count
Total Test Cases	18
Passed	To be recorded
Failed	To be recorded
Pending Verification	18 before execution
Pass Rate	To be calculated after testing

Overall Test Result: Pending final execution and verification.

The implementation should be considered fully verified only after the relevant test cases have been executed and the actual results documented.

6. ACCEPTANCE CRITERIA

ID	Acceptance Criterion	Verification Method	Status
AC-001	The password form is hidden when the dashboard initially loads.	UI inspection	Pending verification
AC-002	Clicking Change Password opens the form.	Functional testing	Pending verification
AC-003	Clicking the toggle button again closes the form.	Functional testing	Pending verification
AC-004	The form contains current password, new password, and confirmation fields.	UI inspection	Pending verification
AC-005	Required-field validation is enforced.	Frontend and API testing	Pending verification

AC-006	Password strength requirements are enforced.	Validation testing	Pending verification
AC-007	The confirmation password must match the new password.	Validation testing	Pending verification
AC-008	The current password is verified before the password is changed.	API testing	Pending verification
AC-009	The new password is hashed before storage.	Code and database verification	Pending verification
AC-010	Only the authenticated student can change their own password through this route.	Authorization testing	Pending verification
AC-011	Appropriate success and error messages are displayed.	UI and API testing	Pending verification
AC-012	Existing login and student profile functionality remain operational.	Regression testing	Pending verification
AC-013	The existing database schema is reused without a new table or column.	Schema inspection	Met by design; environment verification pending
AC-014	The form can be cancelled without changing the password.	UI and API testing	Pending verification

7. IMPLEMENTATION EVIDENCE

Screenshots and other supporting evidence should be inserted into this section before final submission.

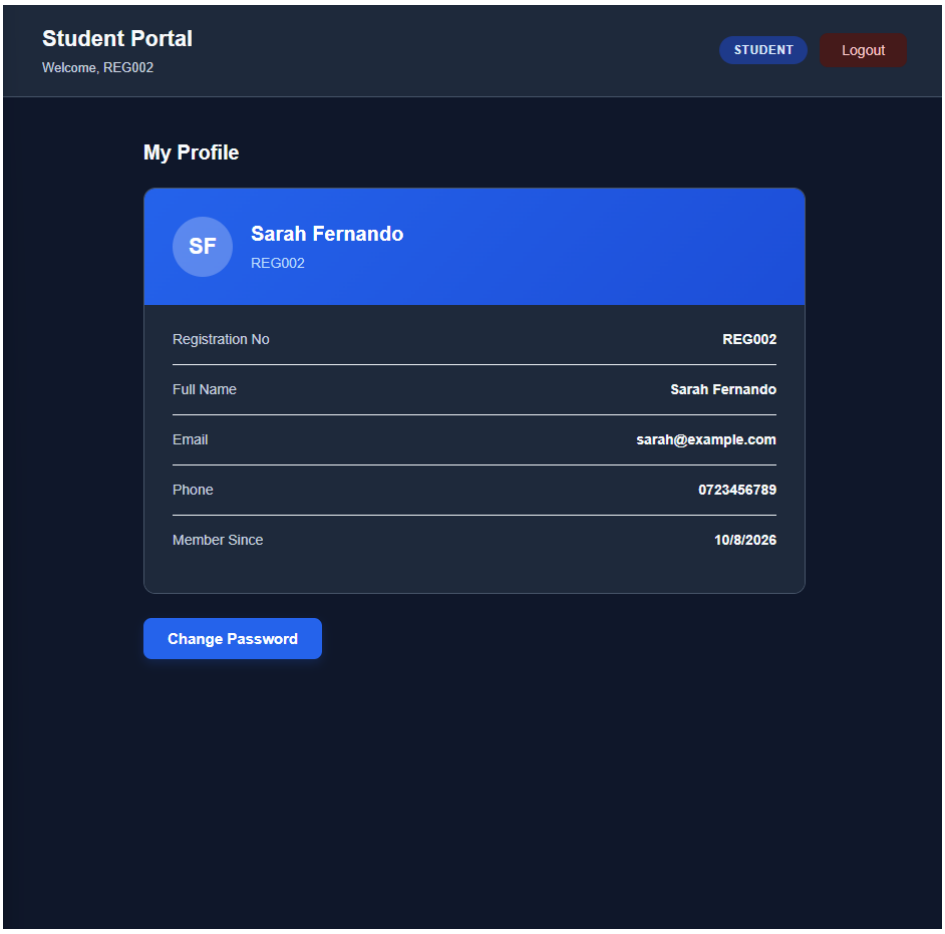
7.1 Evidence 01 – Student Dashboard

Related Acceptance Criterion: AC-001

Description: Insert a screenshot showing the student profile and Change Password button while the password change form is hidden.

Figure 1: Student Dashboard with the Change Password button.

[Insert Screenshot Here]



7.2 Evidence 02 – Password Change Form

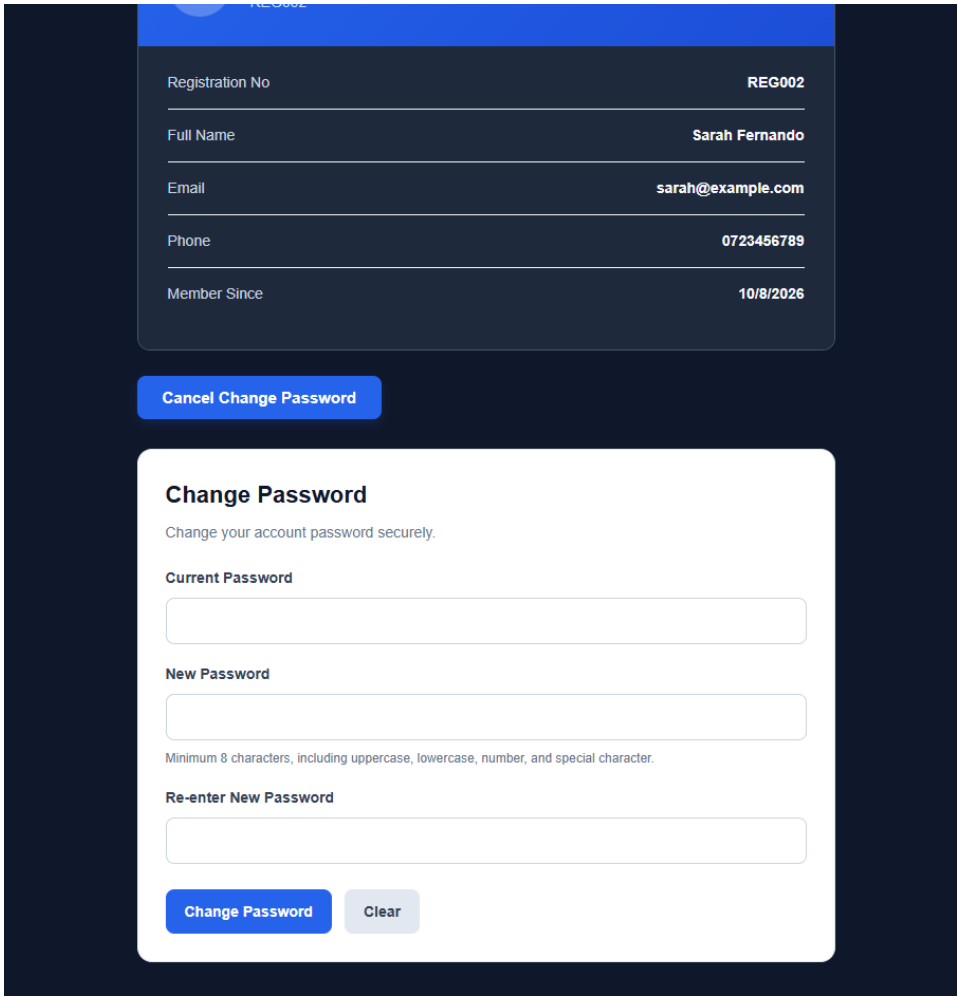
Related Acceptance Criteria: AC-002 and AC-004

Description: Insert a screenshot showing the password change form after the student clicks the Change Password button.

The screenshot should show the current password, new password, confirmation password, submission button, and Cancel button.

Figure 2: Password change form displayed on the Student Dashboard.

[Insert Screenshot Here]



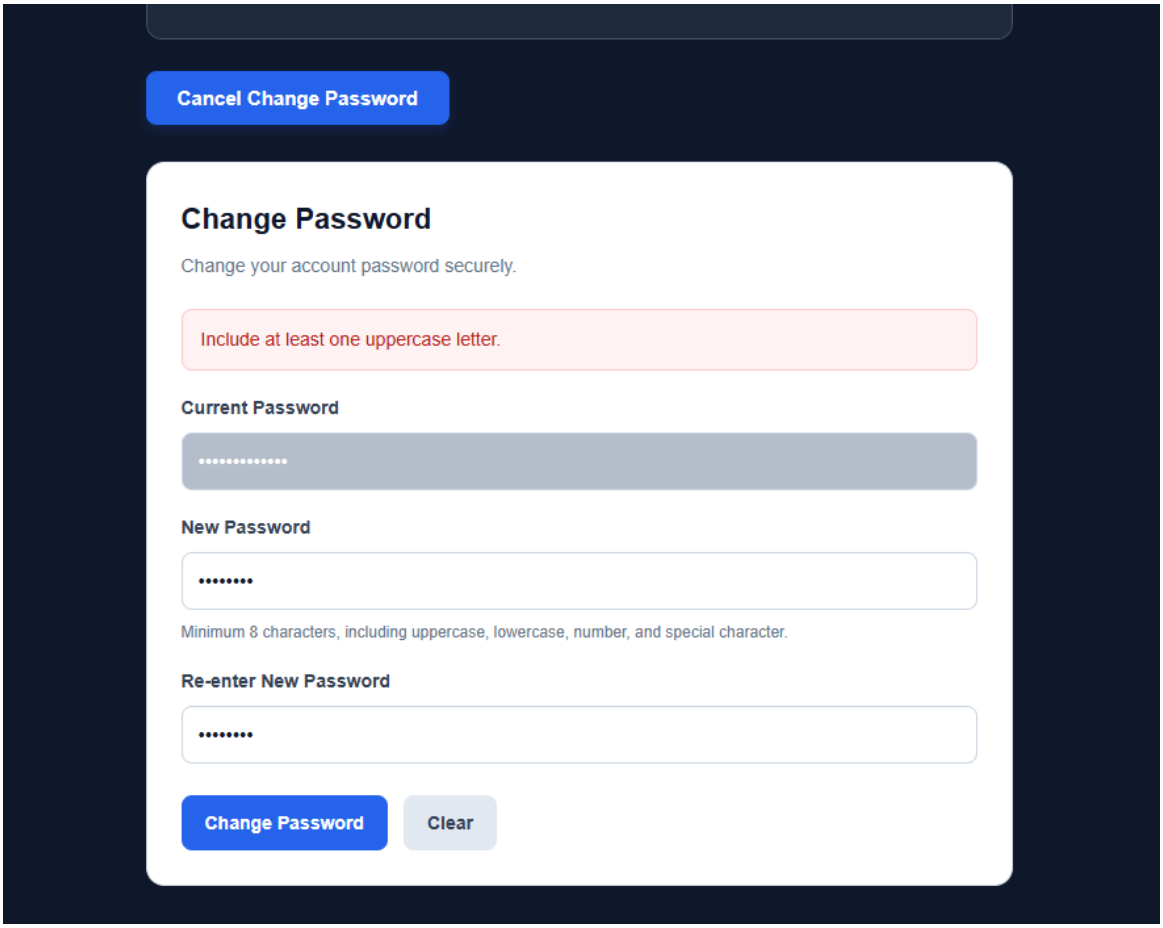
7.4 Evidence 04 – Password Validation

Related Acceptance Criteria: AC-005, AC-006, and AC-007

Description: Insert screenshots showing validation messages for a weak password and mismatched confirmation password.

Figure 4: Password strength and confirmation validation.

[Insert Screenshot Here]



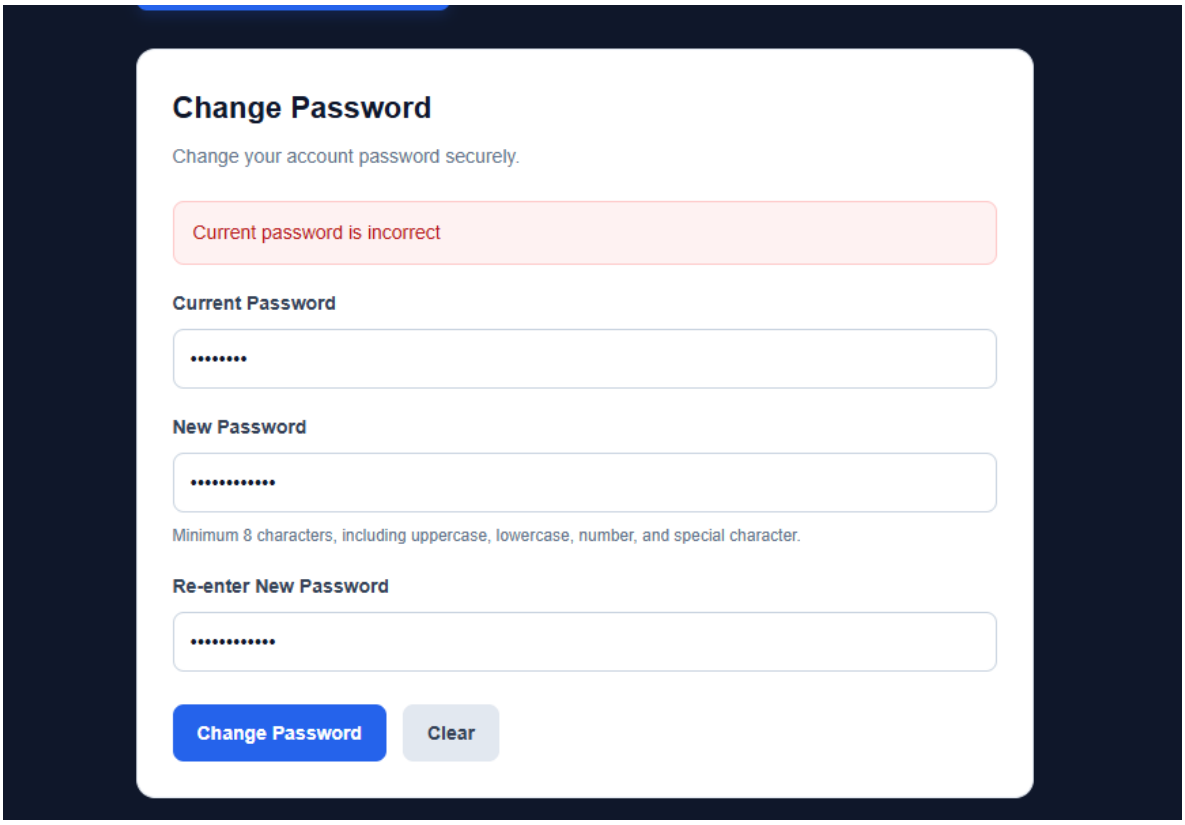
7.5 Evidence 05 – Incorrect Current Password

Related Acceptance Criterion: AC-008

Description: Insert a screenshot showing that the system rejects a password change attempt when the current password is incorrect.

Figure 5: Password change rejected because the current password is incorrect.

[Insert Screenshot Here]



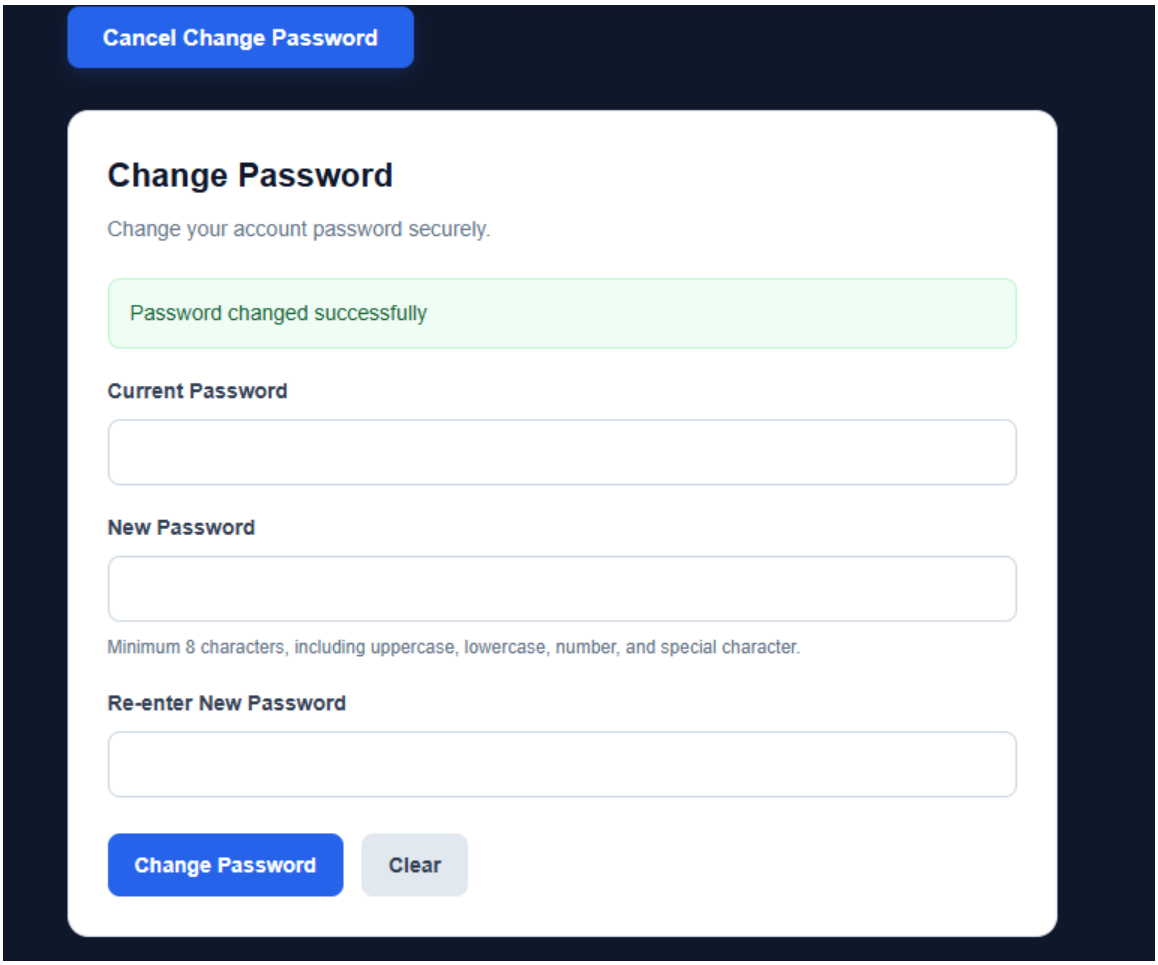
7.6 Evidence 06 – Successful Password Change

Related Acceptance Criteria: AC-009 and AC-011

Description: Insert a screenshot showing the success message after the student submits a valid current password and a valid new password.

Figure 6: Successful student password change.

[Insert Screenshot Here]



8. RESULTS AND DISCUSSION

8.1 Implementation Results

The frontend and backend changes required for CR-020 have been implemented according to the specified requirements.

The Student Dashboard includes a Change Password button, and the password change form is controlled through a visibility state. The form provides fields for the current password, new password, and confirmation password.

The backend provides the authenticated PATCH /api/students/me/password endpoint. It validates submitted values, verifies the current password using bcryptjs, hashes the new password, and updates the existing users.password_hash field.

The implementation reuses the existing database schema and authentication middleware. No additional database table or column is required.

However, functional correctness, authorization behaviour, successful password changes, and regression compatibility must still be confirmed through actual test execution.

8.2 Issues and Limitations

The implementation has the following limitations:

1. The feature depends on the existing authentication middleware, student-role authorization, API service, and user model being configured correctly.
2. Students must know their current password to change it through this facility.
3. Client-side validation alone is insufficient; backend validation must always be enforced.
4. The existing database must contain the users.password_hash field.
5. The feature does not maintain a password-change history or dedicated audit log.
6. The feature does not provide a password-reset workflow for students who have forgotten their current passwords.
7. Existing JWT tokens and active sessions are not automatically revoked after a password change. If immediate session invalidation is required, a separate session-revocation mechanism should be implemented.
8. Test results, screenshots, and final acceptance status must be updated based on actual execution in the development environment.

9. CONCLUSION

CR-020 introduces a self-service password change facility for student accounts in the Student Management System.

The Student Dashboard has been enhanced with a Change Password button that displays the password change form only when requested. Students can enter their current password, provide a new password, confirm it, and cancel the operation without changing their credentials.

The frontend performs validation to improve usability, while the backend validates the submitted information, verifies the current password, and securely hashes the new password before updating the database.

The new endpoint is protected by authentication middleware and student-role authorization. The authenticated student's identity is used to identify the account, preventing the request from selecting another student's account through a supplied user ID.

The implementation reuses the existing database schema, including the users.password_hash field, and does not require additional tables or columns. Existing login, student profile, and administrator functionality are retained in the code.

The implementation work is completed. Final acceptance remains subject to execution of the test cases, verification of the acceptance criteria, and collection of supporting screenshots.

10. FINAL IMPLEMENTATION STATUS

Item	**Result**
Requirements	Implemented in code; verification pending
Frontend Implementation	Completed
Backend Implementation	Completed
Database Schema Changes	Not required
Password Validation	Implemented; test verification pending
Authentication and Authorization	Existing middleware reused; verification pending
Password Hashing	Implemented using bcryptjs
Form Visibility Toggle	Implemented
Success and Error Handling	Implemented
Testing	Pending actual execution
Acceptance Criteria	Pending final verification
Existing Functionality	Regression verification pending
Final Status	Implementation completed; acceptance pending verification

STUDENT DECLARATION

Student Name: S.Hindusha

Student ID: JFICT2426

Date: 13 October 2026

Signature: S.Hindusha

END OF REPORT